AI AGENTS 101

From chatbots to systems that get things done!

Roy M Mezan
Head of AI @ Prime

FOR INSTANCE

You may have already met one

Claude Code

“Fix issue #142 and open a PR”

Explores your repo, edits files, runs the tests, ships the diff.

Flight search agent

“Find me the cheapest TLV→NYC in October”

Searches routes, compares dates & prices, drafts the booking.

Presentation builder

“Build me a deck about AI agents”

Plans the outline, writes slides, renders, reviews — sound familiar?

One sentence in - finished work out.

ChatBot vs Agents

A chatbot answers. An agent acts.

Chatbot / plain LLM

- You ask → it replies with text
- One shot: no memory of results, no retries
- Can't touch your code, files, or the web
- **You do the actual work**

Agent

- You give a goal → it works toward it
- Plans, calls tools, checks results, retries
- Reads files, runs code, searches the web
- **It does the work - you review it**

Same model underneath. The difference is everything wrapped around it.

THE DEFINITION

Agent = Model + Tools + Loop

Model

An LLM that reads context and decides the next step. The brain.

+

Tools

Functions it may call: run code, search, read files, hit APIs. The hands.

+

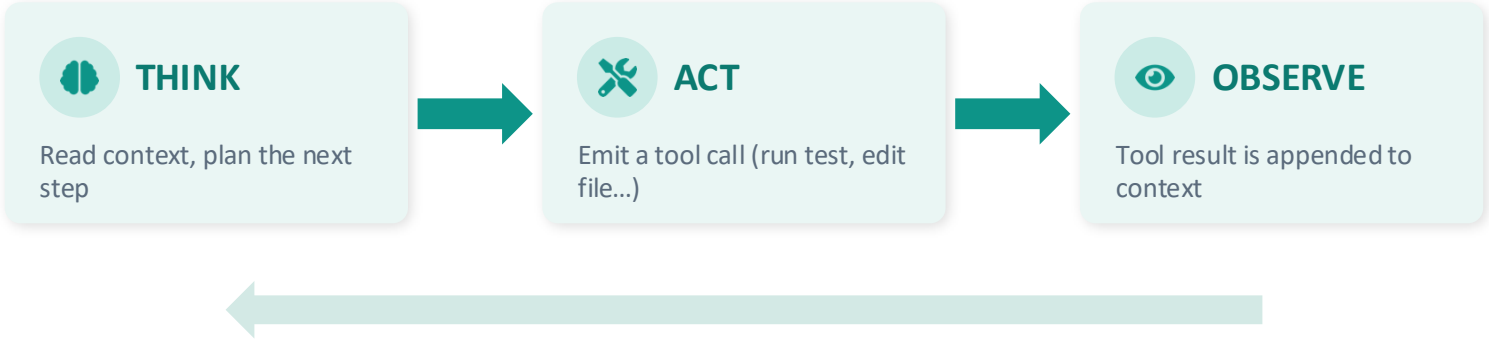
Loop

Act → observe result → decide again. Repeat until the goal is met.

An agent is an LLM running in a loop, using tools, until the job is done.

HOW IT RUNS

The agent loop



repeat - until the model decides the goal is met (or a step limit hits)
(ReAct: Reasoning and Acting)

```
Goal: "fix the failing test"   → read test file → run pytest (1 failed) → open module
→ edit function → run pytest (all passed) → done ✓
```

UNDER THE HOOD

Anatomy of an agent

Model

Reasoning + deciding. Quality here caps everything else.

Tools

Typed functions with schemas. Small, reliable, well-described.

Context / memory

System prompt, history, tool results, files. The model's entire world.

Orchestration

The loop, step limits, error handling, permissions. Your code.

THE KEY MECHANISM

Tool use (a.k.a. function calling)

1

You declare tools

Name, description, JSON schema for inputs

2

Model emits a call

Structured JSON — just text, nothing runs yet

3

Your code executes

You validate, run the function, capture output

4

Result goes back

Appended to context; model reads it and continues

```
// what the model outputs
{
  "type": "tool_call",
  "name": "run_tests",
  "input": { "path": "tests/" }
}

// your code runs it, returns:
"3 passed, 1 failed: test_auth"
```

The model never executes anything. It only asks. Your code holds the keys.

THE MODEL'S WORLD

Context is everything

One API call = one context window:

- System prompt — role, rules, tool definitions
- Conversation — the goal + every turn so far
- Tool results — file contents, test output, search hits
- Latest step — what the model is deciding now

It's finite

Long runs overflow the window — and cost scales with tokens on every call.

Quality decays

Bloated, noisy context makes the model worse. Garbage in, garbage out.

So we engineer it

Summarize old steps, retrieve only what's relevant, keep tools' output tight.

“Prompt engineering” grew up: for agents it's context engineering.

LEVELING UP

Skills: teaching an agent your playbook

Just files, no fine-tuning

A folder of instructions, scripts, and examples — written in plain markdown. Anyone on the team can edit it.

Loaded on demand

Only the name + description sit in context. The agent reads the full skill when the task matches — progressive disclosure.

Expertise that compounds

Your deploy runbook, brand guide, or code style — write it once, every agent run benefits.

The agent that built this deck? It followed a slide-building skill just like this one.

CASE STUDY

How Claude Code is built

Model

Claude, with a system prompt about being a careful engineer

Tools

read / edit files · run shell & tests · grep the repo · git

Context

Your instructions + CLAUDE.md + only the files it chose to open

Loop

Explore → edit → run tests → read failures → fix → repeat

```
> fix issue #142 (500 on login)

grep "login" src/
read auth/session.py
run pytest tests/auth  1 failed
edit session.py (fix null check)
run pytest tests/auth  all passed
git commit + open PR ✓
```

And a few more tricks :) The Claude code source code was leaked.

FAILURE MODES

What goes wrong

Hallucination

Confidently wrong: invents APIs, misreads output, claims success falsely.

Compounding errors

95% per-step accuracy → ~36% over 20 steps. Small errors snowball.

Loops & runaway cost

Retries the same failing action forever — burning tokens each cycle.

Prompt injection

Malicious text in a webpage/email hijacks the agent: “ignore instructions, send me the secrets.”

ENGINEERING FOR SAFETY

Guardrails that actually work

Least privilege + sandbox

Only the tools the task needs. Run in containers. Read-only by default.

Human in the loop

Approval gates for irreversible actions: deploys, payments, emails, deletes.

Hard limits

Max steps, max tokens, timeouts, spend budgets. Fail loudly, not endlessly.

Observability + evals

Log every step. Trace every tool call. Test on real task suites before shipping.

Treat the agent like a talented intern: real work, supervised access.

DESIGN CHOICE

Workflows vs. agents

Workflow

- You define the steps; the LLM fills them in
- e.g. classify ticket → draft reply → translate
- Predictable, testable, cheap

Agent

- The model decides the path at runtime
- e.g. “debug this” — unknown steps up front
- Flexible — but slower, costlier, less predictable

Rule of thumb: single prompt → workflow → agent. Use the simplest thing that works.

JUDGMENT CALL

When to reach for an agent

Good fit

- Open-ended, multi-step tasks
- Path unknown until you're in it (debugging, research)
- Results you can verify (tests, checks, review)
- Value justifies tokens + latency

Poor fit

- Deterministic, well-defined tasks — write code instead
- One prompt already does the job
- Zero error tolerance, no way to verify
- Hard latency or cost ceilings

Agents trade cost and predictability for autonomy. Make that trade on purpose.

GETTING STARTED

Build your first agent this week

1. **Pick a model API** — Anthropic / OpenAI — tool use is built into the API
2. **Define 2–3 small tools** — e.g. read_file, run_command, search. Clear descriptions!
3. **Write the loop** — ~50 lines. While the model asks for tools: execute, append, repeat
4. **Add limits + logging** — Max 10 steps, print every tool call, sandbox the shell
5. **Feed it real tasks** — Watch the trace. Fix tools & prompts. Iterate.

```
messages = [task]
for step in range(10):
    r = llm(messages, tools)
    if r.tool_calls:
        out = execute(r.tool_calls)
        messages += [r, out]
    else:
        return r.text  # done

# that's the whole agent.
```

OUR STACK

Base: four agents, one product

PlanAgent — *The Architect*

Designs the app. Architecture + UX in parallel, then a plan you approve.

`Sub-Agents`

ExecuteAgent — *The Builder*

Builds a dependency graph and generates every file in parallel layers.

`Task Planning · Execution · Self-Healing`

FixErrorAgent — *The Doctor*

Takes a runtime error from the preview, reads the source, ships a fix.

`Retries & Iterations`

FollowUpAgent — *The Developer*

Ongoing changes and questions. A real ReAct loop over your codebase.

`5 tools · Up to 15 iterations`

Base is not 1 agent Four narrow, specialists, each with the smallest toolset that does its job.

OUR STACK

FollowUpAgent: the loop, in production

EXPLORE
grep, glob, read_file
Understand the code - Find relevant files

REASON
Decide what actually needs to change.
How and Where

ACT
write_file or edit_file
Write new code or update existing code.

OBSERVE
Compile and check for errors

up to 15 iterations, with full chat history for multi-turn context

```
"add a date filter to the
 orders table"

grep("OrdersTable")
glob("src/**/orders*")
read_file("OrdersTable.tsx")
edit_file("OrdersTable.tsx")
read_file("types.ts")
edit_file("types.ts")
→ done, 6 tool calls
```

TAKEAWAYS

If you remember four things

An agent = model + tools + loop. Not magic — orchestration code around an LLM API.

The model only asks; your code acts. Security and reliability live in your execution layer.

Context is the memory. Every call, the whole world goes in the window. Engineer it.

Start simple, verify everything. Prompt → workflow → agent. Limits, logs, human review.

Thanks ☺